Add one to a number represented by LL

```
#include <bits/stdc++.h>
using namespace std;

//Definition of
//Single linked list
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
```

```
    {
        val = data1;
        next = next1;
    }
};

class Solution {
public:
    //Function to reverse the linked list
    ListNode* reverseList(ListNode* head) {
        // Initialize pointers
        ListNode* prev = NULL;
        ListNode* current = head;
        ListNode* next = NULL;

        while (current != NULL) {
            // Store next node
            next = current->next;
            // Reverse the link
            current->next = prev;
            // Move prev to current
```

```
        prev = current;
        // Move current to next
        current = next;
    }

    return prev;
}
//Function to add one to Linked List
ListNode *addOne(ListNode *head) {
    //Reverse the linked list
    head = reverseList(head);

    //Create a dummy node
    ListNode* current = head;
    //Initialize carry with 1
    int carry = 1;

    while (current != NULL) {
        /*Sum the current node's value
        and the carry*/
        int sum = current->val + carry;
```

```
//Update carry
carry = sum / 10;
//Update the node's value
current->val = sum % 10;

/*If no carry left
break the loop*/
if (carry == 0) {
    break;
}

/*If we've reached the end of the list
there's still a carry,
add a new node with the carry value*/
if (current->next == NULL && carry != 0) {
    current->next = new ListNode(carry);
    break;
}

// Move to the next node
current = current->next;
```

```
        }

        // Reverse the list
        head = reverseList(head);

        // New head
        return head;
    }
};

//Function to print the linked list
void printList(ListNode* head) {
    while (head != NULL) {
        cout << head->val << " ";
        head = head->next;
    }
    cout << endl;
}

int main() {
    //Creation of Linked List
```

```
    ListNode* head1 = new ListNode(1);
    head1->next = new ListNode(2);
    head1->next->next = new ListNode(3);

    //Solution instance
    Solution solution;
    head1 = solution.addOne(head1);
    cout << "Result after adding one: ";
    printList(head1);

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

// Definition of singly linked list
struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(NULL) {}
    ListNode(int data1) : val(data1), next(NULL) {}
```

```
    ListNode(int data1, ListNode *next1) :
val(data1), next(next1) {}
};

class Solution {
public:
    // Helper function to add one to the linked list
    int addHelper(ListNode* temp) {
        /*If the list is empty
        return a carry of 1*/
        if(temp == NULL)
            return 1;
        /*Recursively call addHelper
        For the next node*/
        int carry = addHelper(temp->next);
        /*Add the carry
        to the current node's value*/
        temp->val += carry;
        /*If the current node's value
        is less than 10
        no further carry is needed*/
        if(temp->val < 10)
```

```
        return 0;
    /* If the current node's value is 10 or more
     set it to 0 and return a carry of 1*/
     temp->val = 0;
     return 1;
}

ListNode *addOne(ListNode *head) {
    /*Call the helper function
    to start the addition process*/
    int carry = addHelper(head);
    /*If there is a carry left
    after processing all nodes
    add a new node at the head */
    if(carry == 1) {
        ListNode* newNode = new ListNode(1);
        /*Link the new node to the current head*/
        newNode->next = head;
        /*Update the head to the new node*/
        head = newNode;
    }
```

```
        // Return the head
        return head;
    }
};

//Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;
    // Traverse the linked list and print each node's value
    while (temp != nullptr) {
        std::cout << temp->val << " ";
        temp = temp->next;
    }
    std::cout << std::endl;
}

int main() {
    //Create a linked list with values 9, 9, 9 (999 + 1 = 1000)
    ListNode* head = new ListNode(9);
    head->next = new ListNode(9);
```

```
    head->next->next = new ListNode(9);

    //Print the original linked list
    cout << "Original Linked List: ";
    printLinkedList(head);

    //Add one to the linked list
    Solution sol;
    head = sol.addOne(head);

    //Print the modified linked list
    cout << "Linked List after adding one: ";
    printLinkedList(head);



    return 0;
}
```

Find Middle of Linked List

```
#include <bits/stdc++.h>
using namespace std;
```

```
//Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
};
```

```
class Solution {
public:
    // Function to get the middle node of linked list
    ListNode* middleOfLinkedList(ListNode* head) {
        ListNode* temp = head;
        int count = 0;

        // Traverse the linked list
        while(temp != NULL) {
            count += 1; // Increment the count by one
            temp = temp-> next;
        }

        int midPosition = (count)/2 + 1;

        ListNode* middleNode = head;
        for(int i = 1; i < midPosition; i++) {
            middleNode = middleNode -> next;
        }
```

```
        return middleNode;
    }
};

// Utility Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;

    // Traverse the linked list and print each node's value
    while (temp != nullptr) {
        cout << temp->val << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Creating a simple linked list
    ListNode* head = new ListNode(1);
    ListNode* second = new ListNode(2);
    ListNode* third = new ListNode(3);
```

```
    ListNode* fourth = new ListNode(4);
    ListNode* fifth = new ListNode(5);

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;

    // Creating an object of Solution class
    Solution sol;

    // Function call to get the middle node of linked list
    ListNode* middleNode = sol.middleOfLinkedList(head);

    printLinkedList(head);
    cout << "The middle node is: " << middleNode->val << endl;

    return 0;
}
```

```
#include <bits/stdc++.h>
using namespace std;

//Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
```

```
        next = next1;
    }
};

class Solution {
public:
    // Function to get the middle node of linked list
    ListNode* middleOfLinkedList(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        // Until the fast pointers reaches NULL or the
last node
        while(fast != NULL && fast->next != NULL) {
            // Move slow pointer by one step
            slow = slow-> next;

            // Move fast pointer by two steps
            fast = fast->next-> next;
        }

        return slow;
```

```
    }
};

// Utility Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;

    // Traverse the linked list and print each node's value
    while (temp != nullptr) {
        cout << temp->val << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Creating a simple linked list
    ListNode* head = new ListNode(1);
    ListNode* second = new ListNode(2);
    ListNode* third = new ListNode(3);
    ListNode* fourth = new ListNode(4);
```

```
    ListNode* fifth = new ListNode(5);

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;

    // Creating an object of Solution class
    Solution sol;

    // Function call to get the middle node of linked list
    ListNode* middleNode = sol.middleOfLinkedList(head);

    printLinkedList(head);
    cout << "The middle node is: " << middleNode->val << endl;

    return 0;
}
```

Delete the middle node in LL

```
#include <bits/stdc++.h>

using namespace std;

// Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
```

```
    {
        val = data1;
        next = next1;
    }
};

class Solution {
public:
    // Function to delete middle node of linked list
    ListNode* deleteMiddle(ListNode* head) {
        /* Edge case: if the list is empty
         * or has only one node, return null */
        if (head == nullptr || head->next == nullptr) {
            delete head;
            return nullptr;
        }

        // Temporary node to traverse
        ListNode* temp = head;

        // Variable to store number of nodes
```

```
int n = 0;

/* Loop to count the number of nodes
in the linked list */
while (temp != nullptr) {
    n++;
    temp = temp->next;
}

// Index of the middle node
int middleIndex = n / 2;

// Reset temporary node
// to beginning of linked list
temp = head;

/* Loop to find the node
just before the middle node */
for (int i = 1; i < middleIndex; i++) {
    temp = temp->next;
}
```

```
        // If the middle node is found
        if (temp->next != nullptr) {
            // Create pointer to the middle node
            ListNode* middle = temp->next;

            // Adjust pointers to skip middle node
            temp->next = temp->next->next;

            /* Free the memory allocated
             * to the middle node */
            delete middle;
        }

        // Return head
        return head;
    }
};

// Function to print the linked list
void printLL(ListNode* head) {
```

```
    ListNode* temp = head;
    while (temp != nullptr) {
        cout << temp->val << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Creating a sample linked list:
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);
    head->next->next->next->next = new
ListNode(5);

    // Display the original linked list
    cout << "Original Linked List: ";
    printLL(head);

    // Deleting the middle node
```

```
    Solution solution;
    head = solution.deleteMiddle(head);

    // Displaying the updated linked list
    cout << "Updated Linked List: ";
    printLL(head);

    return 0;
}
#include <bits/stdc++.h>

using namespace std;

// Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
```

```
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
};

class Solution {
public:
    // Function to delete the middle node of a linked list
    ListNode* deleteMiddle(ListNode* head) {
        /* If the list is empty or has only one node,
         * return NULL as there is no middle node to delete */
```

```
        if (head == NULL || head->next == NULL) {
            return NULL;
        }

        // Initialize slow and fast pointers
        ListNode* slow = head;
        ListNode* fast = head;
        fast = head->next->next;

        // Move 'fast' pointer twice as fast as 'slow'
        while (fast != NULL && fast->next != NULL) {
            slow = slow->next;
            fast = fast->next->next;
        }

        // Delete the middle node by skipping it
        slow->next = slow->next->next;
        return head;
    }
};
```

```
// Function to print the linked list
void printLL(ListNode* head) {
    ListNode* temp = head;
    while (temp != NULL) {
        cout << temp->val << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Creating a sample linked list:
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);
    head->next->next->next->next = new
ListNode(5);

    // Display the original linked list
    cout << "Original Linked List: ";
    printLL(head);
```

```
    // Deleting the middle node
    Solution solution;
    head = solution.deleteMiddle(head);

    // Displaying the updated linked list
    cout << "Updated Linked List: ";
    printLL(head);

    return 0;
}
```

Check if LL is palindrome or not

```
#include <bits/stdc++.h>

using namespace std;

// Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
```

```
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
};


class Solution {
public:
    bool isPalindrome(ListNode* head) {
```

```
/*Create an empty stack
to store values*/
stack<int> st;

/*Initialize temporary pointer
to the head of the linked list*/
ListNode* temp = head;

/*Traverse the linked list
and push values onto the stack*/
while (temp != NULL) {
    /*Push the data from
    the current node onto the stack*/
    st.push(temp->val);

    // Move to the next node
    temp = temp->next;
}

/*Reset temporary pointer
back to the head of
```

```
the linked list*/
temp = head;

/*Compare values by popping
from the stack and checking
\against linked list nodes*/
while (temp != NULL) {
    if (temp->val != st.top()) {
        /*If values don't match,
        it's not a palindrome*/
        return false;
    }

    // Pop the value
    st.pop();

    /*Move to the next node
    in the linked list*/
    temp = temp->next;
}
```

```
        /*If all values match,
        it's a palindrome*/
        return true;
    }
};

/*Function to print the linked list*/
void printLinkedList(ListNode* head) {
    ListNode* temp = head;
    while (temp != nullptr) {
        cout << temp->val << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    /*Create a linked list with values 1, 5, 2, 5, and 1
(15251, a palindrome)*/
    ListNode* head = new ListNode(1);
    head->next = new ListNode(5);
    head->next->next = new ListNode(2);
```

```
    head->next->next->next = new ListNode(5);
    head->next->next->next->next = new
ListNode(1);

    // Print the original linked list
    cout << "Original Linked List: ";
    printLinkedList(head);

    // Check if the linked list is a palindrome
    Solution solution;
    if (solution.isPalindrome(head)) {
        cout << "The linked list is a palindrome." <<
endl;
    } else {
        cout << "The linked list is not a palindrome."
<< endl;
    }

    return 0;
}
#include <bits/stdc++.h>
using namespace std;
```

```
// Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
```

```
};


class Solution {
private:
    /* Function to reverse a linked list
       using the iterative approach */
    ListNode* reverseLinkedList(ListNode* head) {
        // Initialize previous pointer as NULL
        ListNode* prev = NULL;

        // Initialize current pointer as head
        ListNode* curr = head;

        // Traverse the list until the end
        while (curr != NULL) {

            // Temporarily store the next node
            ListNode* nextNode = curr->next;

            // Reverse the link direction
```

```
            curr->next = prev;

            // Move 'prev' one step forward
            prev = curr;

            // Move 'curr' one step forward
            curr = nextNode;
        }

        // 'prev' will now point to the new head
        return prev;
    }

public:
    bool isPalindrome(ListNode* head) {
        /* Check if the linked list is empty
           or has only one node */
        if (head == NULL || head->next == NULL) {
            // It's a palindrome by definition
            return true;
        }
```

```
        /* Initialize two pointers, slow and fast,
           to find the middle of the linked list */
        ListNode* slow = head;
        ListNode* fast = head;

        /* Traverse the linked list using the
           slow and fast pointer approach to
           locate the middle node */
        while (fast->next != NULL && fast->next->next
!= NULL) {

            // Move slow pointer one step
            slow = slow->next;

            // Move fast pointer two steps
            fast = fast->next->next;
        }

        /* Reverse the second half of the linked list
           starting from the node after the middle */
```

```
        ListNode* newHead = reverseLinkedList(slow->next);

        // Pointer to the first half
        ListNode* first = head;

        // Pointer to the reversed second half
        ListNode* second = newHead;

        /* Compare nodes from both halves
           one by one to check for palindrome */
        while (second != NULL) {

            // If mismatch found, not a palindrome
            if (first->val != second->val) {

                // Restore the original list before returning
                reverseLinkedList(newHead);

                return false;
            }
```

```
            // Move both pointers one step ahead
            first = first->next;
            second = second->next;
        }

        /* Restore the second half of the linked list
           to its original order */
        reverseLinkedList(newHead);

        // All values matched, it's a palindrome
        return true;
    }
};


/*Function to print the linked list*/
void printLinkedList(ListNode* head) {
    ListNode* temp = head;
    while (temp != nullptr) {
        cout << temp->val << " ";
```

```
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    /*Create a linked list with values 1, 5, 2, 5, and 1 (15251, a palindrome)*/
    ListNode* head = new ListNode(1);
    head->next = new ListNode(5);
    head->next->next = new ListNode(2);
    head->next->next->next = new ListNode(5);
    head->next->next->next->next = new ListNode(1);

    // Print the original linked list
    cout << "Original Linked List: ";
    printLinkedList(head);

    // Check if the linked list is a palindrome
    Solution solution;
    if (solution.isPalindrome(head)) {
```

```
        cout << "The linked list is a palindrome." <<
endl;
    } else {
        cout << "The linked list is not a palindrome."
<< endl;
    }

    return 0;
}
```

Find the intersection point of Y LL

```
#include <iostream>
#include <unordered_set>

using namespace std;

// Definition of singly linked list
struct ListNode {
    int val;
    ListNode *next;
    ListNode() {
        val = 0;
        next = NULL;
```

```
    }
    ListNode(int data1) {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1) {
        val = data1;
        next = next1;
    }
};

class Solution {
public:
    // Function to find the intersection node of two
linked lists
    ListNode *getIntersectionNode(ListNode
*headA, ListNode *headB) {
        // Create a hash set to store the nodes
        //Of the first list
        unordered_set<ListNode*> nodes_set;

        //Traverse the first linked list
```

```
        //And add all its nodes to the set
        while (headA != NULL) {
            nodes_set.insert(headA);
            headA = headA->next;
        }

        //Traverse the second linked list
        //And check for intersection
        while (headB != NULL) {
            // If a node from the second list is found in
the set,
            // It means there is an intersection
            if (nodes_set.find(headB) !=
nodes_set.end()) {
                return headB;
            }
            headB = headB->next;
        }

        // No intersection found, return NULL
        return NULL;
    }
```

```
};

// Utility function to insert a node at the end of the linked list
void insertNode(ListNode* &head, int val) {
    // Create a new node with the given value
    ListNode* newNode = new ListNode(val);

    // If the list is empty, set the new node as the head
    if (head == NULL) {
        head = newNode;
        return;
    }

    // Otherwise, traverse to the end of the list
    ListNode* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    // Insert the new node at the end of the list
```

```
    temp->next = newNode;
}

// Utility function to print the linked list
void printList(ListNode* head) {
    // Traverse the list
    while (head->next != NULL) {
        // Print the value of each node followed by an
arrow
        cout << head->val << "->";
        head = head->next;
    }
    // Print the value of the last node
    cout << head->val << endl;
}

int main() {
    // Creation of the first list
    ListNode* head1 = NULL;
    insertNode(head1, 1);
    insertNode(head1, 3);
    insertNode(head1, 1);
```

```
    insertNode(head1, 2);
    insertNode(head1, 4);

    // Create an intersection
    ListNode* intersection = head1->next->next->next;

    // Creation of the second list
    ListNode* head2 = NULL;
    insertNode(head2, 3);
    head2->next = intersection;

    // Printing the lists
    cout << "List1: ";
    printList(head1);
    cout << "List2: ";
    printList(head2);

    // Checking if an intersection is present
    Solution sol;
    ListNode* answerNode = sol.getIntersectionNode(head1, head2);
```

```
    if (answerNode == NULL) {
        cout << "No intersection\n";
    } else {
        cout << "The intersection point is " <<
answerNode->val << endl;
    }

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

// Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
```

```
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
};


class Solution {
public:
    ListNode *getIntersectionNode(ListNode
*headA, ListNode *headB) {
        ListNode *temp1 = headA;
        ListNode *temp2 = headB;
        int n1 = 0, n2 = 0;

        // Get the length of first linked list
```

```
        while(temp1 != NULL) {
            n1++;
            temp1 = temp1-> next;
        }

        // Get the length of second linked list
        while(temp2 != NULL) {
            n2++;
            temp2 = temp2-> next;
        }

        // Traverse the longer list and bring the
pointers to same level
        if(n1 < n2) return collisionPoint(headA, headB,
n2 - n1);

        // otherwise
        return collisionPoint(headB, headA, n1 - n2);
    }

private:
    // Function to get the collision point
```

```
    ListNode* collisionPoint(ListNode
*smallerListHead, ListNode *longerListHead, int
len) {
        ListNode *temp1 = smallerListHead;
        ListNode *temp2 = longerListHead;

        // Adjust the pointers to same level
        for(int i = 0; i < len; i++) temp2 = temp2-> next;

        while(temp1 != temp2) {
            temp1 = temp1-> next;
            temp2 = temp2-> next;
        }

        return temp1;
    }
};


// Utility function to insert a node at the end of the
linked list
void insertNode(ListNode* &head, int val) {
```

```
    // Create a new node with the given value
    ListNode* newNode = new ListNode(val);
    // If the list is empty, set the new node as the
head
    if (head == NULL) {
        head = newNode;
        return;
    }
    // Otherwise, traverse to the end of the list
    ListNode* temp = head;
    while (temp->next != NULL) temp = temp->next;
    // Insert the new node at the end of the list
    temp->next = newNode;
}

// Utility function to print the linked list
void printList(ListNode* head) {
    // Traverse the list
    while (head->next != NULL) {
        // Print the value of each node followed by an
arrow
        cout << head->val << "->";
```

```
        head = head->next;
    }
    // Print the value of the last node
    cout << head->val << endl;
}

int main() {
    // Creation of the first list
    ListNode* head1 = NULL;
    insertNode(head1, 1);
    insertNode(head1, 3);
    insertNode(head1, 1);
    insertNode(head1, 2);
    insertNode(head1, 4);

    // Create an intersection
    ListNode* intersection = head1->next->next->next;

    // Creation of the second list
    ListNode* head2 = NULL;
    insertNode(head2, 3);
```

```
    head2->next = intersection;

    // Printing the lists
    cout << "List1: ";
    printList(head1);
    cout << "List2: ";
    printList(head2);

    // Checking if an intersection is present
    Solution sol;
    ListNode* answerNode =
sol.getIntersectionNode(head1, head2);
    if (answerNode == NULL)
        cout << "No intersection\n";
    else
        cout << "The intersection point is " <<
answerNode->val << endl;

    return 0;
}
#include <bits/stdc++.h>
using namespace std;
```

```
// Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
```

```
};



class Solution {
public:
    // Function to find the intersection node of two linked lists
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        // Edge case
        if(headA == NULL || headB == NULL) return NULL;

        //Initialize two pointers to traverse the lists
        ListNode *d1 = headA;
        ListNode *d2 = headB;

        //Traverse both lists until the pointers meet
while (d1 != d2) {
    d1 = (d1 == NULL) ? headB : d1->next;
    d2 = (d2 == NULL) ? headA : d2->next;
}
```

```
return d1;



        //Return the intersection node
        return d1;
    }
};



// Utility function to insert a node at the end of the linked list
void insertNode(ListNode* &head, int val) {
    // Create a new node with the given value
    ListNode* newNode = new ListNode(val);
    // If the list is empty, set the new node as the head
    if (head == NULL) {
        head = newNode;
        return;
    }
    // Otherwise, traverse to the end of the list
    ListNode* temp = head;
```

```
    while (temp->next != NULL) temp = temp->next;
    // Insert the new node at the end of the list
    temp->next = newNode;
}

// Utility function to print the linked list
void printList(ListNode* head) {
    // Traverse the list
    while (head->next != NULL) {
        // Print the value of each node followed by an
arrow
        cout << head->val << "->";
        head = head->next;
    }
    // Print the value of the last node
    cout << head->val << endl;
}

int main() {
    // Creation of the first list
    ListNode* head1 = NULL;
    insertNode(head1, 1);
```

```
    insertNode(head1, 3);
    insertNode(head1, 1);
    insertNode(head1, 2);
    insertNode(head1, 4);

    // Create an intersection
    ListNode* intersection = head1->next->next->next;

    // Creation of the second list
    ListNode* head2 = NULL;
    insertNode(head2, 3);
    head2->next = intersection;

    // Printing the lists
    cout << "List1: ";
    printList(head1);
    cout << "List2: ";
    printList(head2);

    // Checking if an intersection is present
    Solution sol;
```

```
    ListNode* answerNode =
sol.getIntersectionNode(head1, head2);
    if (answerNode == NULL)
        cout << "No intersection\n";
    else
        cout << "The intersection point is " <<
answerNode->val << endl;

    return 0;
}
Detect a loop in LL
#include <bits/stdc++.h>
using namespace std;

// Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
```

```
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
};

class Solution {
public:
    // Function to detect a loop in the linked list
    bool hasCycle(ListNode *head) {
        // Initialize a pointer 'temp'
        // At the head of the linked list
        ListNode* temp = head;
```

```
// Create a set to keep track of
// Encountered nodes
std::unordered_set<ListNode*> nodeSet;

// Traverse the linked list
while (temp != nullptr) {
    // If the node is already in the
    // Set, there is a loop
    if (nodeSet.find(temp) != nodeSet.end()) {
        return true;
    }
    // Store the current node
    // In the set
    nodeSet.insert(temp);

    // Move to the next node
    temp = temp->next;
}

// If the list is successfully traversed
```

```
        // Without a loop, return false
        return false;
    }
};

// Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;
    // Traverse the linked list and print each node's value
    while (temp != nullptr) {
        std::cout << temp->val << " ";
        temp = temp->next;
    }
    std::cout << std::endl;
}

int main() {
    // Create a sample linked list
    // With a loop for testing

    ListNode* head = new ListNode(1);
```

```
    ListNode* second = new ListNode(2);
    ListNode* third = new ListNode(3);
    ListNode* fourth = new ListNode(4);
    ListNode* fifth = new ListNode(5);

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;
    // Create a loop
    fifth->next = third;

    Solution sol;
    // Check if there is a loop
    // In the linked list
    if (sol.hasCycle(head)) {
        cout << "Loop detected in the linked list." <<
endl;
    } else {
        cout << "No loop detected in the linked list."
<< endl;
    }
```

```
    // Clean up memory (free the allocated nodes)
    delete head;
    delete second;
    delete third;
    delete fourth;
    delete fifth;

    return 0;
}
#include <iostream>

// Definition of singly linked list
struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(NULL) {}
    ListNode(int data1) : val(data1), next(NULL) {}
    ListNode(int data1, ListNode *next1) :
val(data1), next(next1) {}
};
```

```
class Solution {
public:
    // Function to detect a loop in a linked
    // list using the Tortoise and Hare Algorithm
    bool hasCycle(ListNode *head) {
        // Initialize two pointers, slow and fast,
        // to the head of the linked list
        ListNode *slow = head;
        ListNode *fast = head;

        // Step 2: Traverse the linked list with
        // the slow and fast pointers
        while (fast != nullptr && fast->next != nullptr) {
            // Move slow one step
            slow = slow->next;
            // Move fast two steps
            fast = fast->next->next;

            // Check if slow and fast pointers meet
            if (slow == fast) {
                return true;  // Loop detected
```

```
            }
        }

        // If fast reaches the end of the list,
        // there is no loop
        return false;
    }
};

// Main function to test the Solution
int main() {
    // Create a sample linked list
    // with a loop for testing

    ListNode *head = new ListNode(1);
    ListNode *second = new ListNode(2);
    ListNode *third = new ListNode(3);
    ListNode *fourth = new ListNode(4);
    ListNode *fifth = new ListNode(5);

    head->next = second;
```

```
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;
    // Create a loop
    fifth->next = third;

    // Create an instance of the Solution class
    Solution solution;

    // Check if there is a loop
    // in the linked list
    if (solution.hasCycle(head)) {
        std::cout << "Loop detected in the linked list."
<< std::endl;
    } else {
        std::cout << "No loop detected in the linked
list." << std::endl;
    }

    // Clean up memory (free the allocated nodes)
    delete head;
    delete second;
```

```
    delete third;
    delete fourth;
    delete fifth;

    return 0;
}
```

Find the starting point in LL

```
#include <iostream>
#include <unordered_map>

// Definition of singly linked list
struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(NULL) {}
    ListNode(int data1) : val(data1), next(NULL) {}
    ListNode(int data1, ListNode *next1) :
val(data1), next(next1) {}
};

class Solution {
```

```
public:
    // Function to find the starting point of the loop
    // in a linked list using hashing technique
    ListNode *findStartingPoint(ListNode *head) {
        // Use temp to traverse the linked list
        ListNode* temp = head;

        // hashmap to store all visited nodes
        std::unordered_map<ListNode*, int> mp;

        // Traverse the list using temp
        while(temp != nullptr) {
            // Check if temp has been encountered
again
            if(mp.count(temp) != 0) {
                // A loop is detected hence return temp
                return temp;
            }
            // Store temp as visited
            mp[temp] = 1;
            // Move to the next node
            temp = temp->next;
```

```
        }

        // If no loop is detected, return nullptr
        return nullptr;
    }
};

// Function to create a linked list with a loop for testing
void createTestList(ListNode* &head, int loop_start_val) {
    ListNode* node1 = new ListNode(1);
    ListNode* node2 = new ListNode(2);
    ListNode* node3 = new ListNode(3);
    ListNode* node4 = new ListNode(4);
    ListNode* node5 = new ListNode(5);

    node1->next = node2;
    node2->next = node3;
    node3->next = node4;
    node4->next = node5;
```

```
    head = node1;

    // Create a loop from node5 to the specified node
    ListNode* temp = head;
    while (temp != nullptr && temp->val != loop_start_val) {
        temp = temp->next;
    }
    node5->next = temp; // Creating the loop
}

int main() {
    // Create a sample linked list with a loop
    ListNode* head = nullptr;
    createTestList(head, 2); // Creating a loop starting at node with value 2

    // Create an instance of the Solution class
    Solution solution;

    // Detect the starting point of the loop in the linked list
```

```
    ListNode* loopStartNode =
solution.findStartingPoint(head);


    if (loopStartNode) {
        std::cout << "Loop detected. Starting node of
the loop is: " << loopStartNode->val << std::endl;
    } else {
        std::cout << "No loop detected in the linked
list." << std::endl;
    }


    // Clean up memory (free the allocated nodes)
    ListNode* temp = head;
    std::unordered_map<ListNode*, bool> visited;
    while (temp != nullptr && visited[temp] == false)
{
        visited[temp] = true;
        ListNode* next = temp->next;
        delete temp;
        temp = next;
    }
```

```
    return 0;
}
#include <iostream>

using namespace std;
//Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
```

```
    {
        val = data1;
        next = next1;
    }
};
```

```
class Solution {
public:
//Function to find the first node
//Of the loop in a linked list
    ListNode* findStartingPoint(ListNode* head) {
        //Initialize a slow and fast
        //Pointers to the head of the list
        ListNode* slow = head;
        ListNode* fast = head;

        // Phase 1: Detect the loop
```

```
while (fast != NULL && fast->next != NULL) {

    // Move slow one step
    slow = slow->next;

    // Move fast two steps
    fast = fast->next->next;

    // If slow and fast meet,
    // a loop is detected
    if (slow == fast) {

        // Reset the slow pointer
        // To the head of the list
        slow = head;

        // Phase 2: Find the first node of the loop
        while (slow != fast) {

            // Move slow and fast one step
            // At a time
```

```
                slow = slow->next;
                fast = fast->next;

                // When slow and fast meet again,
                // It's the first node of the loop
            }

            // Return the first node of the loop
            return slow;
        }
    }

    //If no loop is found,
    //Return NULL
    return NULL;
  }
};

int main() {
  // Create a sample linked list with a loop
  ListNode* node1 = new ListNode(1);
```

```
    ListNode* node2 = new ListNode(2);
    node1->next = node2;
    ListNode* node3 = new ListNode(3);
    node2->next = node3;
    ListNode* node4 = new ListNode(4);
    node3->next = node4;
    ListNode* node5 = new ListNode(5);
    node4->next = node5;

    // Make a loop from node5 to node2
    node5->next = node2;

    // Set the head of the linked list
    ListNode* head = node1;

    // Detect the loop in the linked list
    Solution sol;
    ListNode* loopStartNode =
sol.findStartingPoint(head);

    if (loopStartNode) {
```

```
        cout << "Loop detected. Starting node of the
loop is: " << loopStartNode->val << endl;
    } else {
        cout << "No loop detected in the linked list."
<< endl;
    }

    // Clean up memory (free the allocated nodes)
    delete node1;
    delete node2;
    delete node3;
    delete node4;
    delete node5;

    return 0;
}
```

Length of loop in LL

```
#include <bits/stdc++.h>

using namespace std;

// Definition of singly linked list:
```

```
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
};
```

```
class Solution {
public:
    // Function to find length
    int findLengthOfLoop(ListNode *head) {
        // Hashmap to store visited nodes and their
timer values
        unordered_map<ListNode*, int> visitedNodes;

        // Initialize pointer to traverse the linked list
        ListNode* temp = head;

        // Initialize timer
        // to track visited nodes
        int timer = 0;

        // Traverse the linked list
        // till temp reaches nullptr
        while (temp != NULL) {
            // If revisiting a node return difference of
timer values
            if (visitedNodes.find(temp) !=
visitedNodes.end()) {
```

```
            // Calculate the length of the loop
            int loopLength = timer -
visitedNodes[temp];

            // Return length of loop
            return loopLength;
        }
        /* Store the current node
        and its timer value in
        the hashmap */
        visitedNodes[temp] = timer;

        // Move to the next node
        temp = temp->next;

        // Increment the timer
        timer++;
    }

    /** If traversal is completed
     * and we reach the end
     * of the list (null)
```

```
         * means there is no loop */
        return 0;
    }
};

int main() {
    // Create a sample linked list with a loop
    ListNode* head = new ListNode(1);
    ListNode* second = new ListNode(2);
    ListNode* third = new ListNode(3);
    ListNode* fourth = new ListNode(4);
    ListNode* fifth = new ListNode(5);

    // Create a loop from fifth to second
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;

    // This creates a loop
    fifth->next = second;
```

```
    Solution solution;
    int loopLength =
solution.findLengthOfLoop(head);
    if (loopLength > 0) {
        cout << "Length of the loop: " << loopLength
<< endl;
    } else {
        cout << "No loop found in the linked list." <<
endl;
    }

    return 0;
}
#include <bits/stdc++.h>
using namespace std;

// Definition of singly linked list:
struct ListNode
{
    int val;
    ListNode *next;
```

```
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
};

class Solution {
public:
    // Function to find the length of the loop
    int findLength(ListNode* slow, ListNode* fast) {
```

```
        // Count to keep track of nodes encountered in
loop
        int cnt = 1;

        // Move fast by one step
        fast = fast->next;

        // Traverse fast till it reaches back to slow
        while (slow != fast) {
            /* At each node
            increase count by 1
            move fast forward
            by one step */
            cnt++;
            fast = fast->next;
        }

        /* Loop terminates
        when fast reaches slow again
        and the count is returned*/
        return cnt;
    }
```

```
// Function to find the length of the loop
int findLengthOfLoop(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;

    // Traverse the list to detect a loop
    while (fast != nullptr && fast->next != nullptr) {
        // Move slow one step
        slow = slow->next;
        // Move fast two steps
        fast = fast->next->next;

        // If the slow and fast pointers meet
        // there is a loop
        if (slow == fast) {
            // return the number of nodes
            return findLength(slow, fast);
        }
    }
```

```
        /*If the fast pointer
        reaches the end,
        there is no loop*/
        return 0;
    }
};

int main() {
    // Create a sample linked list with a loop
    ListNode* head = new ListNode(1);
    ListNode* second = new ListNode(2);
    ListNode* third = new ListNode(3);
    ListNode* fourth = new ListNode(4);
    ListNode* fifth = new ListNode(5);

    // Create a loop from fifth to second
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;
    fifth->next = second;
```

```
    Solution solution;
    int loopLength =
solution.findLengthOfLoop(head);
    if (loopLength > 0) {
        cout << "Length of the loop: " << loopLength
<< endl;
    } else {
        cout << "No loop found in the linked list." <<
endl;
    }

    return 0;
}
```